

TP — Séance 1 : Optimisation continue et linéaire

Ce TP se déroule dans le notebook *Seance1_TP1_optimisation_continue_lineaire.ipynb* (Python) et dans le classeur *Seance1_TD_TP1_Solveur_Excel.xlsx* (Excel, feuilles 2 et 3). Cette fiche en est le guide papier : elle rappelle les questions et donne les résultats attendus (dans le corrigé). **Le notebook est découpé pas à pas** : chaque petite étape à compléter (# A COMPLETER) est suivie d'une cellule de **vérification automatique** (✓) qui vous dit si votre code est correct avant de passer à la suite aucune solution n'y est donnée à l'avance. **Le classeur Excel** contient, sur chaque feuille, un mode d'emploi Solveur détaillé pas à pas (quelles cellules remplir, comment paramétrer et lancer le Solveur, comment lire le rapport de sensibilité). En plus, vous **écrirez vous-même** un modèle de programmation linéaire en **MiniZinc**, un langage déclaratif que vous retrouverez en Séance 2 pour la programmation par contraintes.

Exercice 1 Descente de gradient à partir de zéro (notebook, Ex. 1)

Rappel de la règle de mise à jour (Notes de cours, §2.1) : $w_{k+1} = w_k - \alpha \nabla J(w_k)$.

Question 1 • Complétez `gradient_descent` dans le notebook : elle doit renvoyer le point final et l'historique complet des itérés.

Question 2 • Appliquez-la au quadratique $J(w_1, w_2) = 3(w_1 - 1)^2 + (w_2 + 2)^2$ depuis $w_0 = (4, 3)$, pour $\alpha = 0,05$ puis $\alpha = 0,3$. Tracez les trajectoires et commentez (indice : conditionnement de la Hessienne).

Exercice 2 Méthodes accélérées : momentum & Nesterov (notebook, Ex. 2)

Question 1 • Implémentez le momentum (heavy-ball) et le gradient accéléré de Nesterov (Notes de cours, §2.2) dans `momentum_gd`.

Question 2 • Sur le même quadratique qu'à l'Exercice 1 ($\alpha = 0,15$), comparez le nombre d'itérations pour atteindre $\|\nabla J\| < 10^{-6}$: descente de gradient simple, momentum ($\beta = 0,8$), Nesterov ($\beta = 0,8$). Le résultat vous surprend-il ?

Exercice 3 Optimisation continue sous contraintes (notebook, Ex. 3)

Considérons minimiser $(x_1 - 3/2)^2 + (x_2 - 1/2)^4$ s.c. $|x_1| + |x_2| \leq 1$.

Question 1 • Résolvez ce problème de trois façons indépendantes et vérifiez qu'elles s'accordent : (a) barrière logarithmique + `scipy.optimize.minimize` non contraint ; (b) `scipy.optimize.minimize` avec contraintes explicites (SLSQP) ; (c) `cvxpy`.

Question 2 • Écrivez les quatre contraintes sous la forme $g_i(x) \geq 0$, identifiez numériquement lesquelles sont actives à l'optimum, résolvez à la main le système de stationnarité KKT pour les multiplicateurs des contraintes actives, et vérifiez vos valeurs dans le notebook.

Exercice 4

Programmation linéaire : mix de production (*notebook Ex. 4 & Excel, feuille 2*)

Avant l'extension de sa gamme avec le modèle Sport (voir la Partie ExedBike plus loin), ExedBike ne fabriquait que deux modèles, Urban et Cargo — c'est cette version simplifiée, déjà résolue à la main en TD1 (Partie ExedBike — Résolution), que vous vérifiez maintenant avec un solveur, **par deux méthodes indépendantes** : Python et le Solveur Excel.

Question 1 • Résolvez le PL avec `scipy.optimize.linprog` et avec PuLP (*notebook*), puis avec le Solveur Excel (classeur, feuille n° 2 — Mix production z). Les trois résultats coïncident-ils ?

Question 2 • Quelles contraintes sont saturées à l'optimum ? Lisez les prix duaux (`prob.constraints[...].pi` en PuLP, `res.ineqlin.marginals` en `linprog`, ou l'onglet rapport de sensibilité du Solveur Excel) et interprétez-les pour un directeur d'usine.

Exercice 5

Le même PL, en MiniZinc — optimisation déclarative

Les notes de cours (§1.3) présentent la programmation linéaire comme un style *déclaratif* : on décrit *quoi* résoudre (variables, objectif, contraintes), pas *comment* le résoudre. PuLP l'illustre déjà en Python. **MiniZinc** pousse cette idée plus loin — c'est un *langage* dédié à la modélisation déclarative, que vous retrouverez en Séance 2 pour la programmation par contraintes (PPC). Une question naturelle : MiniZinc, souvent associé à l'optimisation combinatoire (entiers), sait-il aussi résoudre un PL *continu* ? **Oui** — le langage admet des variables `var float`, et certains solveurs qu'il pilote (HiGHS, COIN-BC, CPLEX, Gurobi...) sont exactement des solveurs de programmation linéaire continue. **Cette fois, c'est vous qui écrivez le modèle.**

Question 1 • Ouvrez le squelette `Seance1_TP1_minizinc_a_completer.mzn` (fourni) : c'est le PL de l'Exercice 4 (Urban/Cargo, 480h main-d'œuvre, 900kg aluminium, 240h machine, marges 120/200), à écrire en MiniZinc en trois étapes commentées dans le fichier. Complétez, dans l'ordre : (1) les deux variables de décision `var float` ; (2) les trois contraintes de ressources ; (3) l'objectif `solve maximize`. Lancez ensuite votre modèle avec un solveur adapté aux variables continues : `minizinc -solver HiGHS fichier.mzn` (ou le menu déroulant de MiniZinc IDE). Vous devez retrouver *exactement* Urban = 75, Cargo = 30, marge = 15 000 — comme en TD1, Python et Excel. Une fois que ça marche, relancez le **même fichier** avec le solveur *Gecode* par défaut (`minizinc -solver Gecode fichier.mzn`, sans `-solver HiGHS`). Que se passe-t-il ?

Pour aller plus loin (*curiosité facultative, non guidée — fichiers déjà écrits, à lancer tels quels*)

Question 2 • Le modèle précédent ne compte que 2 variables. Passons à l'échelle : ouvrez `exedbike_large.mzn` (fourni, déjà complet — rien à écrire ici), un modèle *générique* qui lit n variantes et m ressources depuis un fichier de données `.dzn` (généré par `generate_instance.py` — catalogue ExedBike étendu à un configurateur tailles × couleurs × options). Exécutez-le sur `large.dzn` ($n = 2000$ variantes, $m = 80$ ressources) avec HiGHS. Puis, sur la *petite* instance `small.dzn` ($n = 15$, $m = 4$ seulement !), comparez HiGHS et Gecode avec `-time-limit 20000`. Qu'observez-vous, et pourquoi l'écart entre les deux solveurs devient-il *plus* spectaculaire, pas moins, sur la petite instance ?

Partie ExedBike

Projet individuel, Partie 1 — à rendre sur GitHub

C'est ici que se termine la Partie 1 de votre mini-projet fil rouge, lancée en TD1. **Rappel du barème (déjà annoncé en TD1, non cumulatif avec cette fiche)** pour le notebook complété (cost model + PL) et la synthèse d'une page ; pour la résolution manuelle du simplexe (Partie ExedBike — Résolution, TD1). Cette section de la fiche ne fait que détailler ce qui est attendu dans le notebook — elle ne crée pas de barème supplémentaire.

Question 1 • Ajustez le modèle de coût $c(q) = k \cdot q^{-\theta}$ sur les données fournies (régression log-log) avec **votre propre** implémentation de la descente de gradient (Exercice 1) — pas un solveur boîte noire. Pourquoi faut-il standardiser $\log q$ avant d'optimiser ?

Question 2 • Formulez et résolvez le PL du mix de production pour la gamme **réelle** à trois produits (Urban, Cargo, Sport — table ci-dessous, identique à celle formulée en TD1, Partie projet) avec PuLP ou `scipy.optimize.linprog`. Reportez le mix optimal, la marge, les contraintes saturées, les prix duaux, et une phrase d'interprétation métier par prix dual.

Ressource	Urban	Cargo	Sport	Disponible
Main-d'uvre (h/unité)	4	6	5	520
Aluminium (kg/unité)	6	15	9	1100
Temps machine (h/unité)	2	2	3	260
Marge (/unité)	120	200	175	—

Livrables (rappel du TD1) : le notebook `Seance1_TP1_optimisation_continue_lineaire.ipynb` complété (Questions 1–2 ci-dessus) et une courte synthèse (≤ 1 page : paramètres du modèle de coût, mix optimal, interprétation des prix duaux pour un lecteur non technique), **commités sur votre dépôt GitHub personnel** avant la Séance 2. Vous pouvez vérifier votre résultat de la Question 2 de façon indépendante avec le Solveur Excel (classeur, feuille n° 3 — Mix production (réel) z) ou avec MiniZinc — ce n'est pas un livrable supplémentaire, seulement une vérification croisée recommandée avant de rendre.